Laboratorio 11 — Hoja de trabajo

Ventas con transacciones, acopios, reportes y CSV

Nombre: Ronald Salvatierra Flores **Fecha:** 20 / 10 / 2026

Comandos: `docker compose up -d --build --wait` · `docker compose exec -T php php verificar.php` (objetivo: 15 de 15). Ventas: operador o admin · Reportes: solo admin.

Parte A — El modelo de datos

1. Tablas ventas, venta_detalle, acopios, productos y socios y sus relaciones.

```
socios 1 ——— N productos          (productos.socio_id → socios.id)
ventas 1 ——— N venta_detalle      (venta_detalle.venta_id → ventas.id, ON DELETE CASCADE)
productos 1 —— N venta_detalle    (venta_detalle.producto_id → productos.id)
socios 1 ——— N acopios            (acopios.socio_id → socios.id)
productos 1 —— N acopios          (acopios.producto_id → productos.id)
usuarios 1 ——— N ventas           (ventas.usuario_id → usuarios.id: quién vendió)
```

Entre ventas y productos hay una relación de muchos a muchos (una venta tiene varios productos y un producto está en muchas ventas); se resuelve con la tabla intermedia venta_detalle. Los acopios registran lo que el socio entrega a la asociación y también apuntan al socio y al producto.

2. ¿Por qué venta_detalle guarda precio_unitario si el producto ya tiene precio?

Porque el precio de un producto puede cambiar con el tiempo. Si el reporte usara el precio actual de productos, las ventas de julio cambiarían de valor cada vez que se actualiza el precio. Guardar el precio del día en cada línea es un dato histórico: el importe de cada venta queda fijo, y los reportes (SUM(cantidad × precio_unitario)) siempre dan el mismo resultado.

3. ¿Por qué la clave primaria de venta_detalle es (venta_id, producto_id)? ¿Para qué sirve ix_ventas_fecha?

La clave compuesta impide que el mismo producto aparezca dos veces en la misma venta (la aplicación suma las cantidades repetidas en una sola línea) y no necesita una columna id artificial. El índice ix_ventas_fecha acelera los reportes, que filtran siempre por rango de fechas (fecha >= desde AND fecha < hasta): MySQL busca en el índice en lugar de recorrer toda la tabla.

Parte B — Transacciones (TODO 1 y 2)

Estado inicial (datos del laboratorio): MIE-500 = 32, CER-1K = 4, PRO-30 = 25 y 96 ventas.

1. Venta de 2 MIE-500 + 99 CER-1K sin transacción: ¿qué error aparece, cuánto stock queda, se crea la venta? ¿Qué problema contable es?

Sin el TODO 1 cada sentencia se confirma sola. El código descontaba primero 2 MIE-500 (stock 32 → 30) y, al restar 99 a CER-1K (stock 4), MySQL rechaza el UPDATE por la restricción CHECK (stock >= 0) y la aplicación responde con un error 500. Pero el descuento de MIE-500 ya estaba guardado: no se crea ninguna venta y aun así faltan 2 unidades de miel. Es una inconsistencia contable: el inventario baja sin una venta que lo respalde. (Razonado a partir del código de partida; con el código ya corregido no se puede reproducir.)

2. Completa el TODO 1. Repite la prueba: ¿qué cambió?

```
$this->bd->beginTransaction();
try {
    …
    $this->bd->commit();
    return $ventaId;
} catch (Throwable $e) {
```

```
    if ($this->bd->inTransaction()) { $this->bd->rollBack(); }
    throw $e;
}
```

Ahora todo es una sola operación: o se guarda la venta completa o no se guarda nada. Al repetir la prueba (2 MIE-500 + 99 CER-1K) la aplicación responde 409 (Figura 1) y el stock queda igual: MIE-500 = 32, CER-1K = 4, PRO-30 = 25 y 96 ventas (Figura 2). El rollback deshizo el descuento de la primera línea.



Figura 1. Venta con 99 CER-1K: 409 «Stock insuficiente de CER-1K para vender 99».

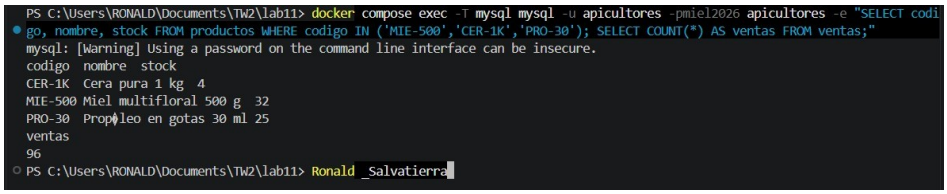


Figura 2. Tras el rollback: stock sin cambios (32, 4 y 25) y 96 ventas.

3. TODO 2: ¿por qué es mejor UPDATE … SET stock = stock - :cantidad WHERE id = :id AND stock >= :minimo que leer con SELECT, comparar en PHP y hacer el UPDATE?

```
UPDATE productos SET stock = stock - :cantidad WHERE id = :id AND stock >= :minimo
-- si rowCount() === 0  →  throw new Conflicto("Stock insuficiente de $codigo para vender $cantidad.")
```

Con SELECT + comparación + UPDATE hay una ventana de tiempo entre leer y escribir: dos vendedoras a la vez ven «quedan 3», las dos venden 3 y el stock queda en −3 (sobreventa). Con la condición dentro del UPDATE, la comprobación y el descuento son una sola operación atómica: MySQL bloquea la fila, la segunda vendedora ve el stock ya descontado y su UPDATE no modifica ninguna fila (rowCount = 0). El verificador comprueba el caso límite: vender 3 CER-1K (hay 4) funciona y una segunda venta de 3 da Conflicto con el stock en 1.

4. ¿Qué responde ahora la aplicación? ¿Por qué 409 y no 500?

Responde 409 Conflict con el mensaje «No se puede completar — Stock insuficiente de CER-1K para vender 99» (Figura 1). 409 significa que la petición es válida pero choca con el estado actual de los datos (no hay suficiente stock); es un error del cliente que puede corregirse. 500 indicaría un fallo inesperado del servidor, que aquí no ocurre.

5. Venta válida (2 MIE-500 + 1 PRO-30) y lectura de RegistroAcopio: ¿qué hace SELECT … FOR UPDATE?

La venta válida se registra como «Venta N.º 97» (Figura 3) y el stock pasa a MIE-500 = 30 y PRO-30 = 24, con 97 ventas (Figura 4); el total es Bs 128,00 (2 × 45 + 1 × 38).



Figura 3. Venta válida: «Venta N.º 97 registrada» (2 MIE-500 + 1 PRO-30).

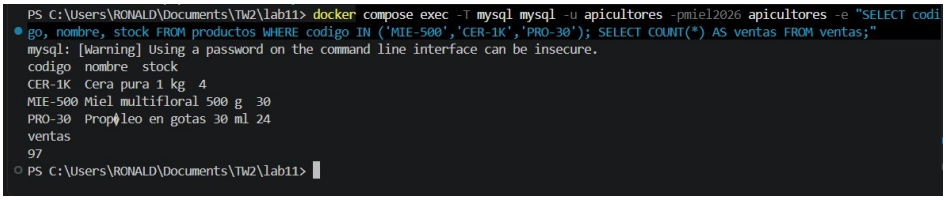


Figura 4. Después de la venta: MIE-500 = 30, PRO-30 = 24 y 97 ventas.

SELECT … FOR UPDATE bloquea la fila del producto hasta el final de la transacción: otra transacción que quiera modificar esa fila espera, así el acopio lee y actualiza el stock sin que nadie lo cambie en medio. El verificador comprueba el acopio: +10 CER-1K sube el stock de 1 a 11 y registra al socio dueño del producto; con cantidad 0 lanza DatosInvalidos y no altera el stock (Figura 6).

Parte C — Reportes con SQL (TODO 3, 4 y 5)

1. TODO 3: ventas por mes de julio a septiembre.

```
SELECT DATE_FORMAT(v.fecha, '%Y-%m') AS mes, COUNT(*) AS ventas, SUM(v.total) AS total
FROM ventas v WHERE v.fecha >= :desde AND v.fecha < :hasta
GROUP BY DATE_FORMAT(v.fecha, '%Y-%m') ORDER BY mes
Mes       Ventas   Total
2026-07      35   Bs 13.637,50
2026-08      26   Bs 14.110,50
2026-09      35   Bs 13.483,50
```

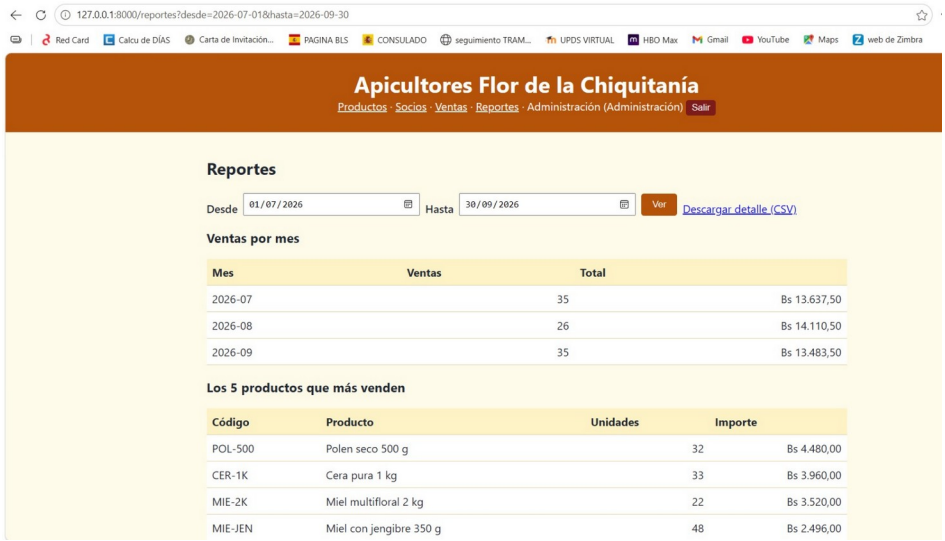


Figura 5. /reportes (administrador): ventas por mes y los productos que más venden.

2. TODO 4: los 5 productos con más importe. ¿Por qué agrupar por p.id y no solo ordenar?

```
SELECT p.codigo, p.nombre, SUM(d.cantidad) AS unidades, SUM(d.cantidad * d.precio_unitario) AS importe
FROM venta_detalle d JOIN ventas v ON v.id = d.venta_id JOIN productos p ON p.id = d.producto_id
WHERE v.fecha >= :desde AND v.fecha < :hasta
GROUP BY p.id, p.codigo, p.nombre ORDER BY importe DESC, p.codigo LIMIT :limite
```

POL-500 (Bs 4.480), CER-1K (3.960), MIE-2K (3.520), MIE-JEN (2.496) y MIE-TOC (2.244). Ordenar sin agrupar devolvería una fila por cada línea de venta; GROUP BY junta todas las líneas de un mismo producto para que SUM calcule el total de ese producto. Se agrupa por p.id (la clave) para no mezclar productos distintos con el mismo nombre. El límite se enlaza con bindValue y PDO::PARAM_INT, como en la paginación.

3. porSocio(): ¿qué diferencia hay entre WHERE y HAVING? ¿Cuántos socios superan Bs 3.000?

WHERE filtra las filas antes de agrupar (aquí, las ventas del período); HAVING filtra los grupos después de calcular el SUM (aquí, los socios cuyo importe total es ≥ 3000). No se puede usar WHERE con un SUM porque en ese momento aún no existe. Superan Bs 3.000 cuatro de los ocho socios: Julián Pesoa (Bs 10.289), Carmen Tomichá (9.761), Hernán Parapaino (8.634) y Mario Surubí (4.681,50).

4. TODO 5: productos sin ventas. ¿Por qué LEFT JOIN … IS NULL y no JOIN?

```
SELECT p.codigo, p.nombre, p.stock FROM productos p
LEFT JOIN (SELECT DISTINCT d.producto_id FROM venta_detalle d JOIN ventas v ON v.id = d.venta_id
           WHERE v.fecha >= :desde AND v.fecha < :hasta) vendidos ON vendidos.producto_id = p.id
WHERE vendidos.producto_id IS NULL ORDER BY p.codigo
```

Un JOIN solo devuelve productos que SÍ tienen coincidencia en las ventas, así que nunca mostraría los que no se vendieron. LEFT JOIN conserva todos los productos y rellena con NULL las columnas del lado derecho cuando no hay venta; el filtro IS NULL deja justamente esos. Resultado: CER-BLO, JAL-30, MIE-OR y VIN-MIE.

5. ¿Por qué el período usa fecha < hasta + 1 día y no fecha <= hasta?

Porque fecha es DATETIME. Con hasta = 2026-09-30 la comparación fecha <= '2026-09-30' equivale a ≤ 2026-09-30 00:00:00 y dejaría fuera las ventas del 30/09 a las 11:05. Usar fecha < '2026-10-01 00:00:00' incluye todo el día. El verificador lo comprueba: las ventas del 30/09 entran con hasta=2026-09-30.

Parte D — Fechas y CSV (TODO 6 y 7)

1. DateTimeImmutable::createFromFormat("!Y-m-d","2026-02-31"): ¿qué fecha da? Completa el TODO 7.

```
if ($fecha === false || $fecha->format('Y-m-d') !== $texto) {
    throw new DatosInvalidos(['general' => "La fecha '$texto' no es válida (formato AAAA-MM-DD)."]);
}
```

Da 2026-03-03: PHP «corre» el 31 de febrero al 3 de marzo en lugar de rechazarlo. El TODO 7 vuelve a formatear la fecha y la compara con el texto recibido; si no coinciden, la fecha era imposible y se responde 422. El verificador comprueba que /reportes?desde=2026-02-31 y desde > hasta dan 422.

2. Descarga el CSV y ábrelo. ¿Se ven bien las tildes y los decimales? ¿Para qué sirven el BOM y el separador ;?

El archivo descargado (ventas_20260701_20260930.csv) empieza con el BOM UTF-8, usa «;» como separador y coma decimal, y tiene una fila por cada línea de detalle: 170 filas más el encabezado, para las 96 ventas del trimestre. El BOM (EF BB BF) le indica a Excel que el archivo es UTF-8, por eso se ven bien las tildes y la ñ («Concepción», «Bálsamo»). El separador «;» es el que Excel usa en

configuración regional española, donde la coma es el separador decimal (52,00); con «,» Excel mezclaría las columnas con los decimales.

```
Venta;Fecha;Cliente;Código;Producto;Cantidad;"Precio unitario";Subtotal
1;"2026-07-01 16:50";"Supermercado El Sol";MIE-JEN;"Miel con jengibre 350 g";12;52,00;624,00
1;"2026-07-01 16:50";"Supermercado El Sol";MIE-PAN;"Miel en panal 400 g";2;95,00;190,00
```

3. Venta con el cliente =HYPERLINK(…): ¿qué haría Excel con esa celda? ¿Cómo lo evita celdaSegura()?

```
$texto = (string) $valor;
if ($texto !== '' && in_array($texto[0], ['=', '+', '-', '@', "\t", "\r"], true)) {
    return "'" . $texto;
}
return $texto;
```

Sin el TODO 6, una celda que empieza con = se interpreta como fórmula: Excel crearía un enlace «Ver factura» hacia un sitio elegido por el atacante (o ejecutaría otras funciones). Es la inyección de fórmulas CSV. celdaSegura() antepone un apóstrofo a los textos que empiezan con =, +, - o @ (o tabulador / retorno), así la celda se trata como texto. El verificador lo comprueba con el cliente =HYPERLINK(…): el CSV exportado contiene '=HYPERLINK y no una fórmula (Figura 6). Los números con decimales se convierten a coma sin pasar por esta regla, para que «-5» o «+3» de un dato numérico no se confundan con una fórmula.

Parte E — Verificación

php verificar.php → objetivo 15 de 15.

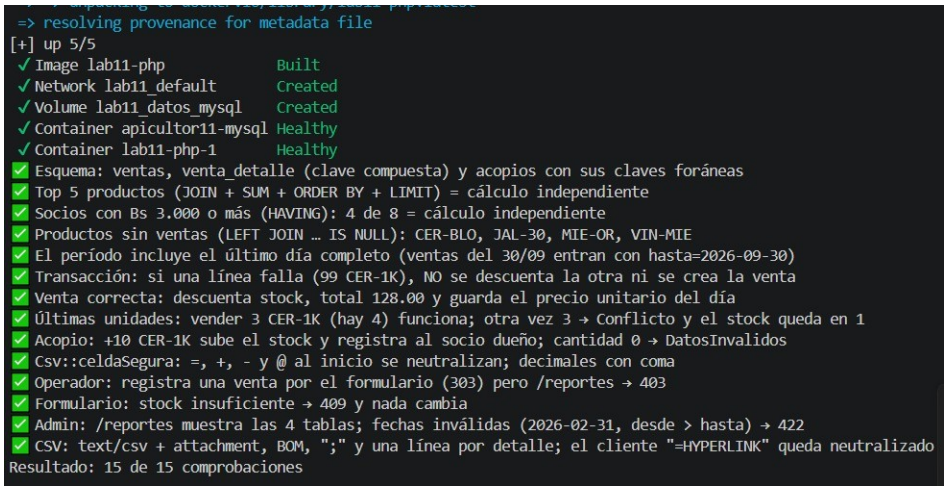


Figura 6. Verificador del laboratorio 11 en Docker: 15 de 15 comprobaciones.

Los reportes se comparan con un cálculo independiente hecho en PHP sobre las filas sin agrupar (mes, top 5, socios con Bs 3.000 o más, productos sin ventas), de modo que un error de SQL no pasa desapercibido. En la prueba de roles, el operador registra la venta (303) pero /reportes responde 403.

Reflexión

¿Qué preguntas de la directiva responde ahora el sistema que antes se discutían «con el cuaderno»?

¿Cuánto vendimos cada mes? (Bs 13.637,50 en julio, 14.110,50 en agosto y 13.483,50 en septiembre). ¿Qué productos se venden más? (POL-500, CER-1K y MIE-2K). ¿Cuánto le corresponde generar a cada socio? (cuatro socios superan Bs 3.000). ¿Qué productos no se

movieron en el trimestre? (CER-BLO, JAL-30, MIE-OR y VIN-MIE, que conviene revisar o promocionar). Además, el inventario ya no se descuadra: una venta o se registra completa o no se registra, y no se vende más de lo que hay.

Propón un reporte nuevo útil para la asociación y escribe su consulta SQL.

Clientes que más compran: sirve para decidir a quién ofrecer descuentos por volumen o fidelizar. Con los datos del laboratorio (jul–sep 2026) el primero es Farmacia Santa Ana, con 19 compras y Bs 8.603.

```
SELECT v.cliente, COUNT(*) AS compras, SUM(v.total) AS total
FROM ventas v
WHERE v.fecha >= :desde AND v.fecha < :hasta
GROUP BY v.cliente
ORDER BY total DESC
LIMIT 5;
Farmacia Santa Ana       19 compras  Bs 8.603,00
Tienda Naturista Vida    15 compras  Bs 6.436,00
Hotel Misiones           12 compras  Bs 5.301,50
Supermercado El Sol      16 compras  Bs 5.110,00
Cliente de mostrador      8 compras  Bs 4.546,00
```

Anexo — Archivos con los TODO resueltos

Archivos del laboratorio con los TODO 1 al 7 resueltos. Además se agregó al docker-compose.yml el servicio php (con su Dockerfile) y name: tw2-lab11.

src/Ventas/RegistroVenta.php

```
<?php
// src/Ventas/RegistroVenta.php — Registra una venta de VARIOS productos como una sola operación
(transacción):
// o se guarda todo (venta, detalle y descuento de stock) o no se guarda nada.
declare(strict_types=1);

namespace Apicultores\Ventas;

use Apicultores\Excepciones\Conflicto;
use Apicultores\Excepciones\DatosInvalidos;
use PDO;
use Throwable;

final class RegistroVenta
{
    public function __construct(private readonly PDO $bd)
    {
    }

    /**
     * @param array<string, int> $lineas código → cantidad
     * @return int id de la venta
     * @throws DatosInvalidos|Conflicto
     */
    public function registrar(array $lineas, string $cliente, int $usuarioId): int
    {
        if ($lineas === [] || trim($cliente) === '') {
            throw new DatosInvalidos(['general' => 'Indica el cliente y al menos un producto con
cantidad.']);
        }
        // TODO 1: todo lo de abajo es UNA operación: beginTransaction() aquí, commit() al final
        //         y, si algo falla, rollBack() en el catch antes de volver a lanzar la excepción.
        $this->bd->beginTransaction();
        try {
            $buscar = $this->bd->prepare('SELECT id, precio FROM productos WHERE codigo = ?');
            // Descuento ATÓMICO: solo resta si alcanza el stock. Si dos ventas compiten por las últimas
unidades,
            // MySQL bloquea la fila y la segunda ve el stock ya descontado (rowCount = 0).
            // TODO 2: el UPDATE solo resta si ALCANZA el stock (AND stock >= :minimo) y, si rowCount()
es 0,
            //         se lanza Conflicto("Stock insuficiente de $codigo para vender $cantidad.").
            $descontar = $this->bd->prepare('UPDATE productos SET stock = stock - :cantidad WHERE id
= :id AND stock >= :minimo');
            $detalle = [];
            $total = 0.0;
            foreach ($lineas as $codigo => $cantidad) {
                $buscar->execute([$codigo]);
                $p = $buscar->fetch() ?: throw new DatosInvalidos(['general' => "No existe el producto
$codigo."]);
```

```
                $descontar->execute(['cantidad' => $cantidad, 'id' => $p['id'], 'minimo' => $cantidad]);
                if ($descontar->rowCount() === 0) {
                    throw new Conflicto("Stock insuficiente de $codigo para vender $cantidad.");
                }
                $detalle[] = [$p['id'], $cantidad, $p['precio']];
                $total += $cantidad * (float) $p['precio'];
            }
            $this->bd->prepare('INSERT INTO ventas (fecha, cliente, usuario_id, total) VALUES (NOW(), ?,
?, ?)')
                ->execute([trim($cliente), $usuarioId, number_format($total, 2, '.', '')]);
            $ventaId = (int) $this->bd->lastInsertId();
            $insertar = $this->bd->prepare('INSERT INTO venta_detalle (venta_id, producto_id, cantidad,
precio_unitario) VALUES (?, ?, ?, ?)');
            foreach ($detalle as [$productoId, $cantidad, $precio]) {
                $insertar->execute([$ventaId, $productoId, $cantidad, $precio]);
            }
            $this->bd->commit();
            return $ventaId;
        } catch (Throwable $e) {
            if ($this->bd->inTransaction()) {
                $this->bd->rollBack();
            }
            throw $e;
        }
    }
}
```

src/Reportes/ReporteVentas.php

```
<?php
// src/Reportes/ReporteVentas.php — Las preguntas de la directiva, respondidas con SQL (JOIN, GROUP BY,
HAVING, LEFT JOIN).
// Todas reciben un período [desde, hasta] con fechas YA validadas (DateTimeImmutable).
declare(strict_types=1);

namespace Apicultores\Reportes;

use DateTimeImmutable;
use PDO;

final class ReporteVentas
{
    public function __construct(private readonly PDO $bd)
    {
    }

    /** Período cerrado en ambos extremos: hasta 2026-09-30 incluye todo ese día (< 2026-10-01). */
    private function periodo(DateTimeImmutable $desde, DateTimeImmutable $hasta): array
    {
        return ['desde' => $desde->format('Y-m-d 00:00:00'), 'hasta' => $hasta->modify('+1 day')-
>format('Y-m-d 00:00:00')];
    }

    /** 1) ¿Cuánto vendimos cada mes? */
    public function porMes(DateTimeImmutable $desde, DateTimeImmutable $hasta): array
    {
```

```
        // TODO 3: una fila por mes con: mes (DATE_FORMAT(v.fecha, '%Y-%m')), ventas (COUNT) y total
(SUM(v.total)),
        //         solo del período (v.fecha >= :desde AND v.fecha < :hasta), agrupado y ordenado por
mes.
        $c = $this->bd->prepare(
            "SELECT DATE_FORMAT(v.fecha, '%Y-%m') AS mes, COUNT(*) AS ventas, SUM(v.total) AS total
             FROM ventas v
             WHERE v.fecha >= :desde AND v.fecha < :hasta
             GROUP BY DATE_FORMAT(v.fecha, '%Y-%m')
             ORDER BY mes");
        $c->execute($this->periodo($desde, $hasta));
        return $c->fetchAll();
    }

    /** 2) ¿Qué productos se venden más? (unidades e importe) */
    public function topProductos(DateTimeImmutable $desde, DateTimeImmutable $hasta, int $limite = 5):
array
    {
        // TODO 4: codigo, nombre, unidades (SUM(d.cantidad)) e importe (SUM(d.cantidad *
d.precio_unitario)):
        //         venta_detalle JOIN ventas JOIN productos, filtrado por período, GROUP BY producto,
        //         ORDER BY importe DESC, p.codigo y LIMIT :limite (bindValue con PDO::PARAM_INT).
        $c = $this->bd->prepare(
            'SELECT p.codigo, p.nombre, SUM(d.cantidad) AS unidades, SUM(d.cantidad * d.precio_unitario)
AS importe
             FROM venta_detalle d
             JOIN ventas v    ON v.id = d.venta_id
             JOIN productos p ON p.id = d.producto_id
             WHERE v.fecha >= :desde AND v.fecha < :hasta
             GROUP BY p.id, p.codigo, p.nombre
             ORDER BY importe DESC, p.codigo
             LIMIT :limite');
        foreach ($this->periodo($desde, $hasta) as $nombre => $valor) {
            $c->bindValue(':' . $nombre, $valor);
        }
        $c->bindValue(':limite', $limite, PDO::PARAM_INT);
        $c->execute();
        return $c->fetchAll();
    }

    /** 3) ¿Cuánto vendió la asociación con los productos de cada socio? Solo los que superan un mínimo
(HAVING). */
    public function porSocio(DateTimeImmutable $desde, DateTimeImmutable $hasta, float $minimo = 0):
array
    {
        $c = $this->bd->prepare(
            'SELECT s.nombre AS socio, s.comunidad, SUM(d.cantidad * d.precio_unitario) AS importe
             FROM venta_detalle d
             JOIN ventas v    ON v.id = d.venta_id
             JOIN productos p ON p.id = d.producto_id
             JOIN socios s    ON s.id = p.socio_id
             WHERE v.fecha >= :desde AND v.fecha < :hasta
             GROUP BY s.id, s.nombre, s.comunidad
             HAVING importe >= :minimo
             ORDER BY importe DESC');
        $c->execute($this->periodo($desde, $hasta) + ['minimo' => $minimo]);
```

```
        return $c->fetchAll();
    }

    /** 4) ¿Qué productos NO se vendieron en el período? (LEFT JOIN … IS NULL) */
    public function sinVentas(DateTimeImmutable $desde, DateTimeImmutable $hasta): array
    {
        // TODO 5: productos (codigo, nombre, stock) que NO aparecen en ninguna venta del período.
        //         Pista: productos p LEFT JOIN (productos vendidos en el período) … WHERE
vendidos.producto_id IS NULL
        $c = $this->bd->prepare(
            'SELECT p.codigo, p.nombre, p.stock
             FROM productos p
             LEFT JOIN (SELECT DISTINCT d.producto_id
                        FROM venta_detalle d
                        JOIN ventas v ON v.id = d.venta_id
                        WHERE v.fecha >= :desde AND v.fecha < :hasta) vendidos ON vendidos.producto_id =
p.id
             WHERE vendidos.producto_id IS NULL
             ORDER BY p.codigo');
        $c->execute($this->periodo($desde, $hasta));
        return $c->fetchAll();
    }

    /** Detalle línea por línea (para exportar a CSV). */
    public function detalle(DateTimeImmutable $desde, DateTimeImmutable $hasta): array
    {
        $c = $this->bd->prepare(
            'SELECT v.id AS venta, DATE_FORMAT(v.fecha, \'%Y-%m-%d %H:%i\') AS fecha, v.cliente,
p.codigo, p.nombre AS producto,
                    d.cantidad, d.precio_unitario, d.cantidad * d.precio_unitario AS subtotal
             FROM venta_detalle d
             JOIN ventas v    ON v.id = d.venta_id
             JOIN productos p ON p.id = d.producto_id
             WHERE v.fecha >= :desde AND v.fecha < :hasta
             ORDER BY v.fecha, v.id, p.codigo');
        $c->execute($this->periodo($desde, $hasta));
        return $c->fetchAll();
    }
}
```

src/Reportes/Csv.php

```
<?php
// src/Reportes/Csv.php — Exportación a CSV que Excel en español abre bien y que no ejecuta fórmulas.
declare(strict_types=1);

namespace Apicultores\Reportes;

final class Csv
{
    /** Genera el contenido CSV (separador ";" y coma decimal, como espera Excel configurado en
español). */
    public static function generar(array $encabezados, array $filas): string
    {
        $flujo = fopen('php://temp', 'r+');
        fwrite($flujo, "\xEF\xBB\xBF");                       // BOM: Excel reconoce UTF-8 (tildes y ñ)
        fputcsv($flujo, $encabezados, ';', '"', '');
```

```
        foreach ($filas as $fila) {
            fputcsv($flujo, array_map(self::celdaSegura(...), array_values($fila)), ';', '"', '');
        }
        rewind($flujo);
        return (string) stream_get_contents($flujo);
    }

    /**
     * Inyección CSV: un cliente llamado "=HYPERLINK(...)" se convertiría en FÓRMULA al abrir el archivo
en Excel.
     * Se antepone un apóstrofo a los textos que empiezan con = + - @ (o tabulador / retorno).
     */
    public static function celdaSegura(mixed $valor): string
    {
        if (is_float($valor) || (is_string($valor) && preg_match('/^-?\d+\.\d+$/', $valor))) {
            return str_replace('.', ',', (string) $valor);     // 45.50 → 45,50
        }
        // TODO 6: si el texto empieza con = + - @ (o tabulador / retorno), antepone un apóstrofo (').
        $texto = (string) $valor;
        if ($texto !== '' && in_array($texto[0], ['=', '+', '-', '@', "\t", "\r"], true)) {
            return "'" . $texto;
        }
        return $texto;
    }
}
```